



Techniki optymalizacji renderowania świata złożonego z voxeli

Dawid Ciesielski¹

Magdalena Długosz¹

Anton Prykhodzka¹

Artur Szymański²

Przemysław Walczak¹

Adam Ziółkowski¹

¹Technologie gier i symulacji komputerowych, FTIMS, Politechnika Łódzka, Polska

²Grafika komputerowa i Multimedia, FTIMS, Politechnika Łódzka, Polska



Wprowadzenie

Gry oparte na **voxelach** umożliwiają modelowanie zniszczalnego środowiska i dużą swobodę interakcji. Jednocześnie wymagają wydajnych technik **renderowania** i zarządzania **geometrią**.

Problem

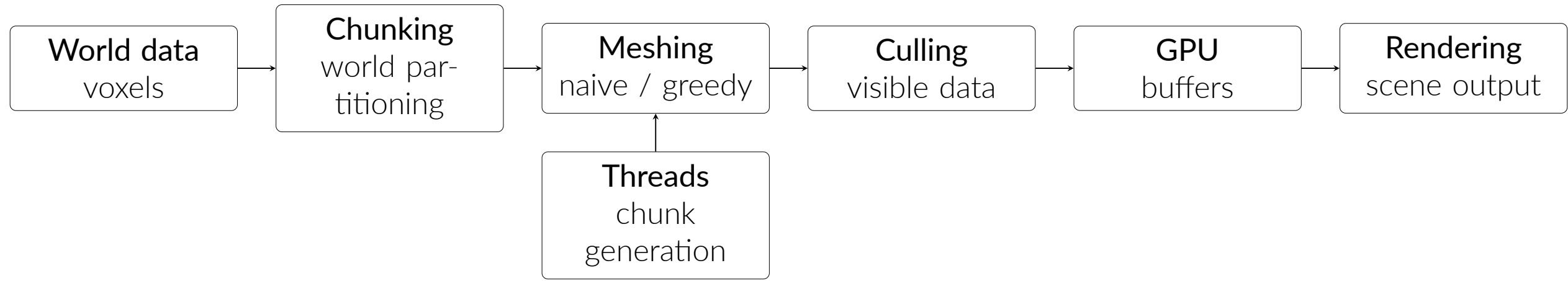
Renderowanie świata voxelowego w czasie rzeczywistym prowadzi do wysokiego obciążenia **CPU** i **GPU**, szczególnie przy dynamicznych zmianach geometrii. Celem projektu jest redukcja kosztów obliczeniowych przy zachowaniu poprawnej reprezentacji sceny.

Zastosowane techniki optymalizacji

- **Chunking** – podział świata na mniejsze fragmenty.
- **Meshing** – redukcja liczby trójkątów generowanych dla voxelowej geometrii.
- **Culling** – odrzucanie niewidocznych elementów sceny.
- **LOD** (Level of Detail) – zmniejszanie szczegółowości odległych obiektów.
- **Multithreading** – generowanie chunków i siatek w osobnych wątkach.

Architektura systemu

System oparty jest na podziale na chunki oraz asynchronicznym generowaniu siatek. Dane świata są przetwarzane w tle, a do renderowania przekazywane są wyłącznie widoczne fragmenty.



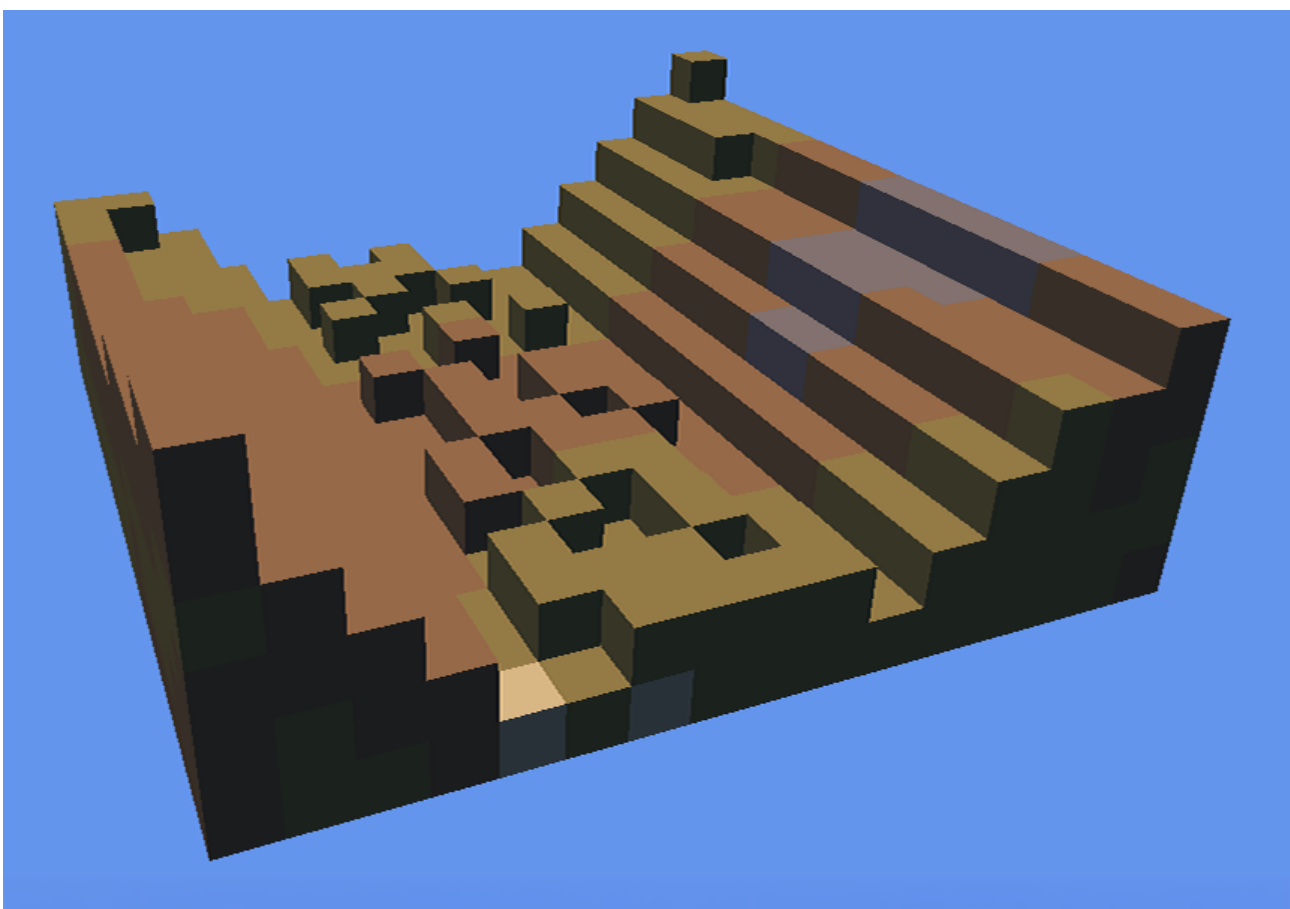
Voxel world processing and rendering pipeline

System przetwarza dane voxelowe w kilku etapach. Świat dzielony jest na **chunki**, dla których generowane są siatki z wykorzystaniem technik optymalizacyjnych. Następnie wykonywany jest proces **cullingu**, a do GPU przekazywane są wyłącznie widoczne dane. Generowanie chunków i siatek odbywa się **równolegle** w osobnych wątkach.

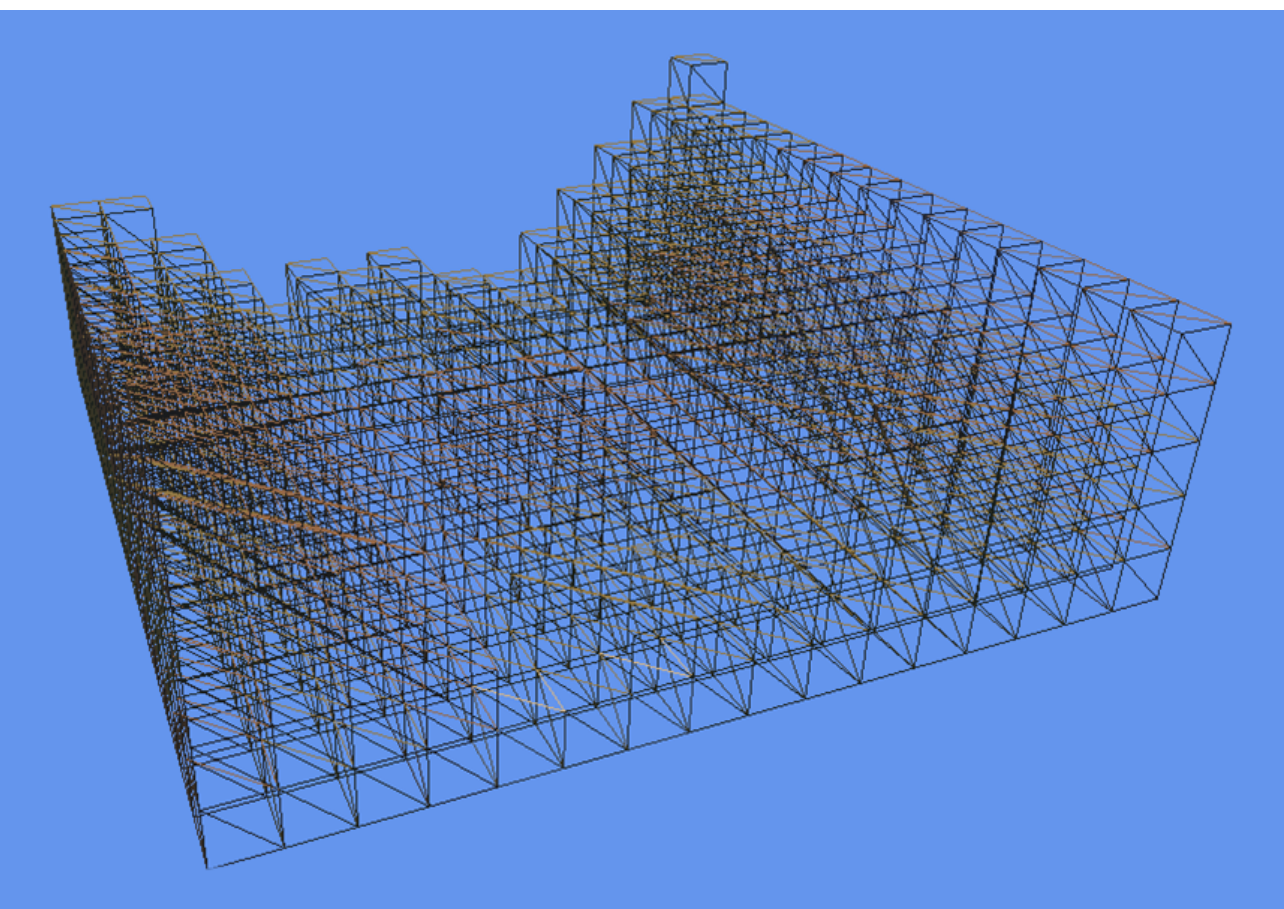
Technologie i środowisko

- **Sprzęt** – Nvidia Geforce GTX 1070 8 GB VRAM, i7-7700k 4.5GHz, 16 GB RAM
- **MonoGame + C#** – implementacja autorskiego silnika.
- **ECS** – podział logiki na encje, komponenty i systemy.
- **HLSL** – własny pipeline renderowania i shadery.
- **Profilowanie** – pomiar FPS, CPU, GPU, draw calli i średniego czasu generowania chunka.

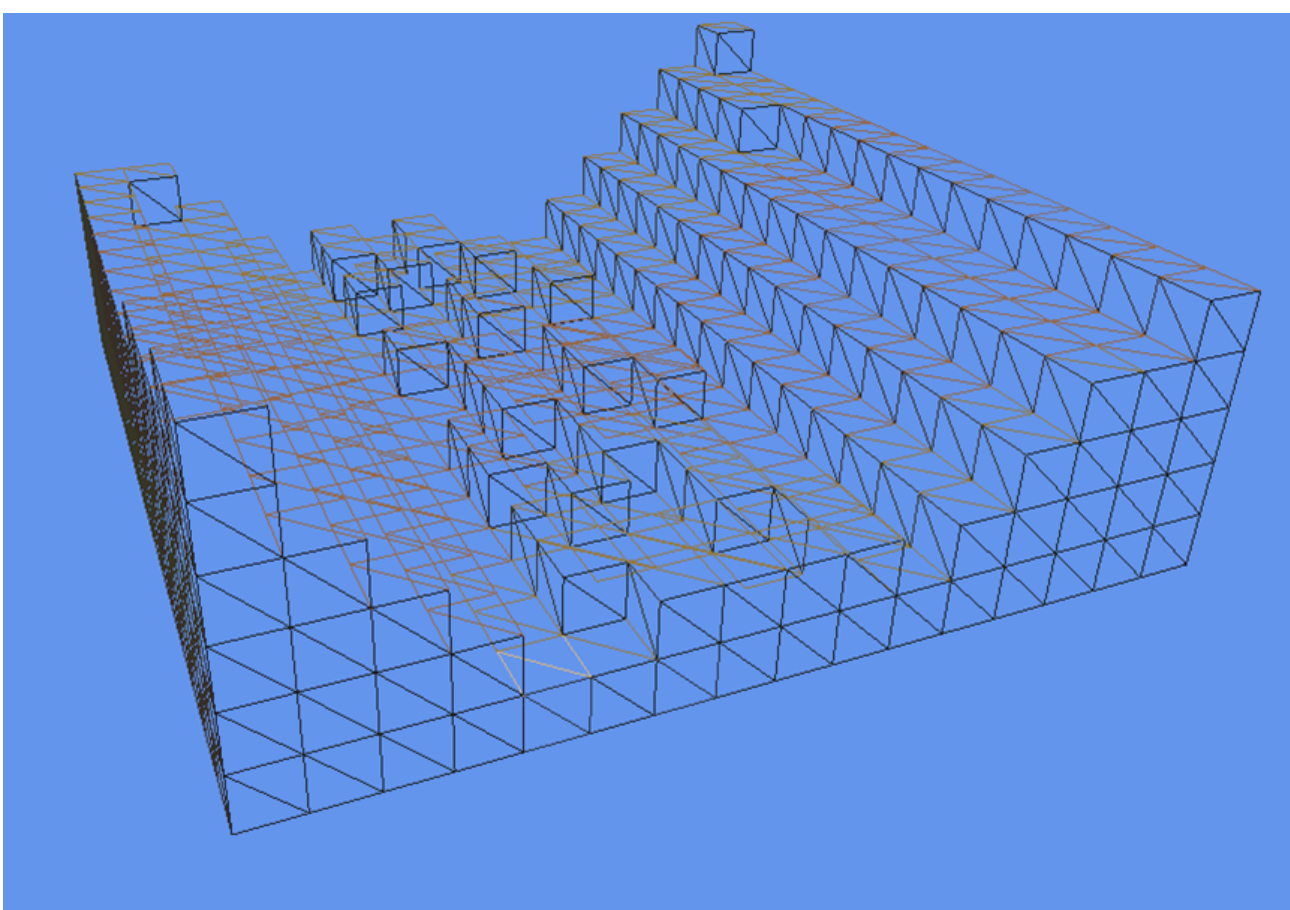
Porównanie metod generowania siatek



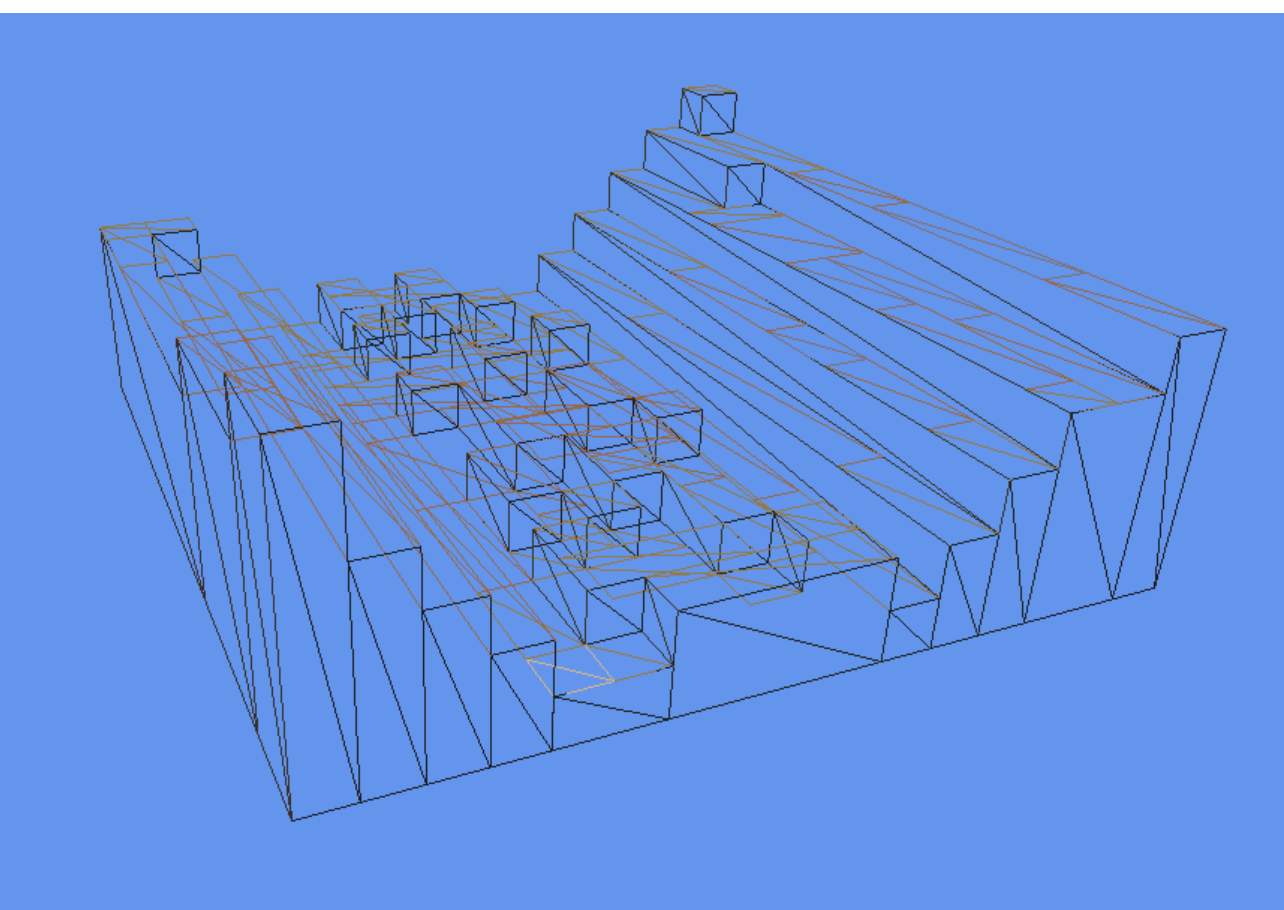
(a) Scena oryginalna



(b) Naive meshing – 7812 trójkątów



(c) Usunięcie ścian wewnętrznych – 1932 trójkąty



(d) Greedy meshing – 410 trójkątów

Przedstawione obrazy pokazują wpływ kolejnych technik optymalizacji na złożoność geometrii. Każdy etap redukuje liczbę trójkątów przy zachowaniu ogólnego kształtu sceny.

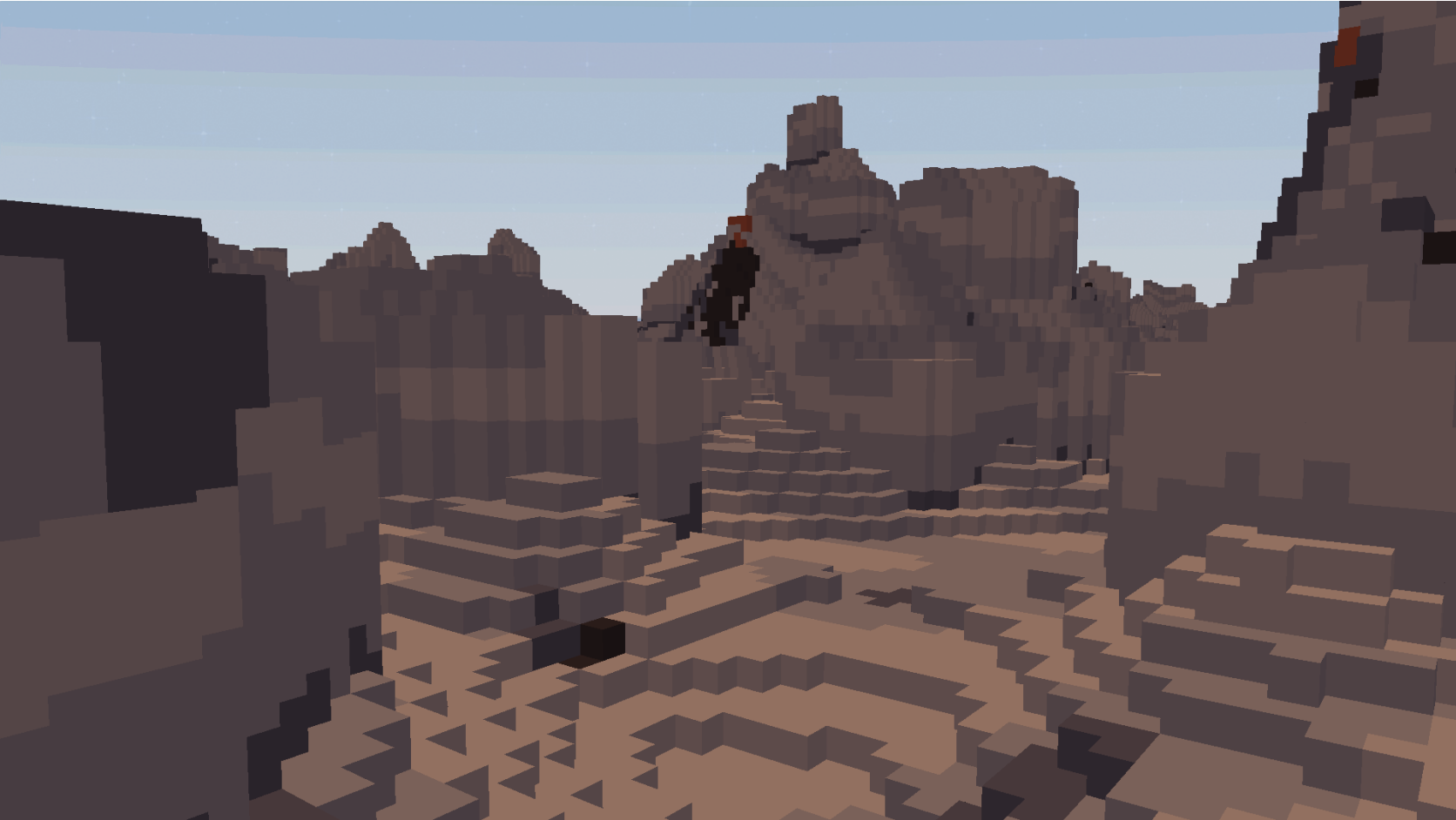
Optymalizacja pamięci voxelowej

Jednym z kluczowych usprawnień było zmniejszenie zajętości pamięci przez pojedynczy voxel.

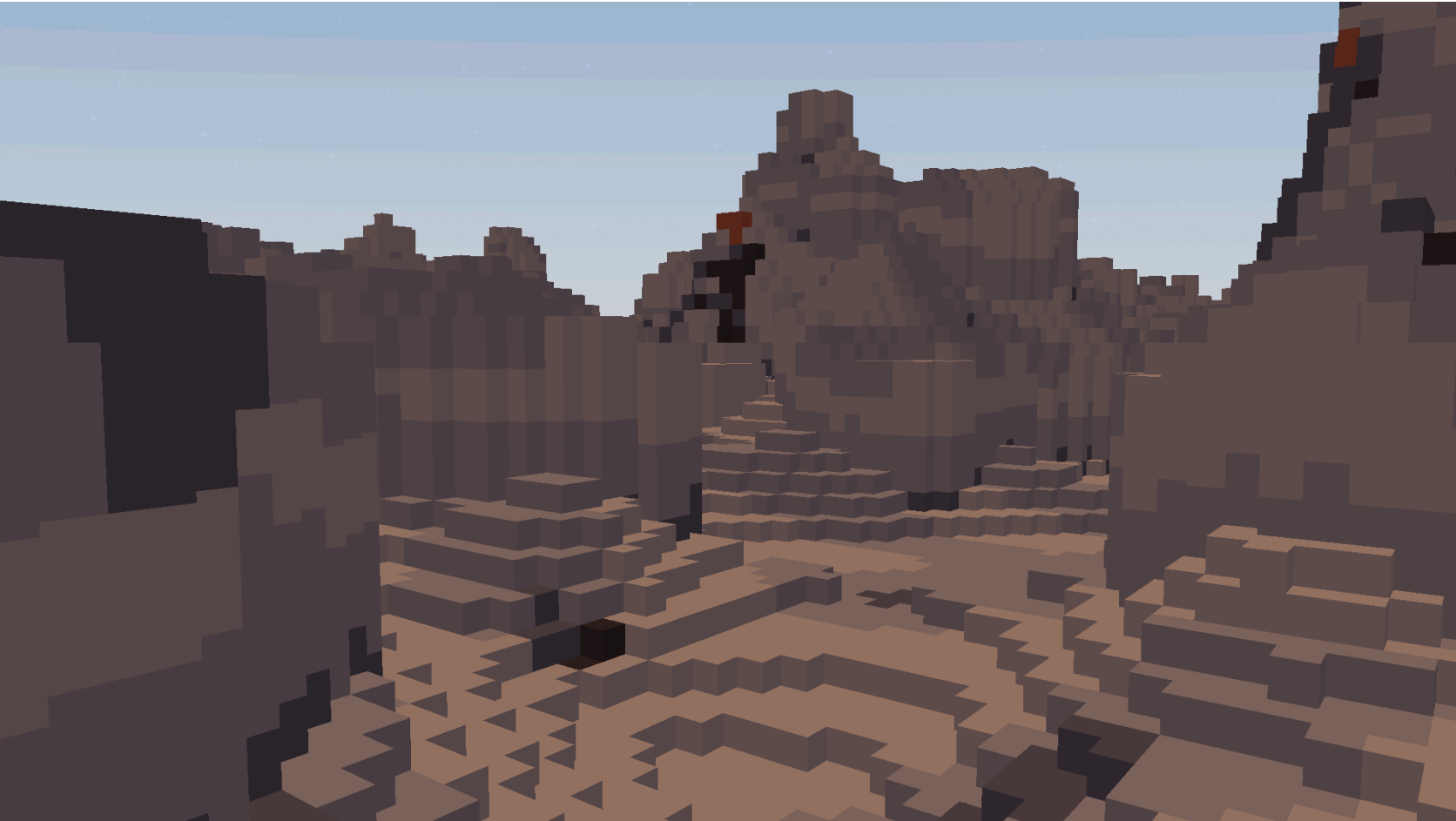
W zoptymalizowanej wersji zastąpiono strukturę **Color + bool (5 byte)** pojedynczym **bajtem**, gdzie bajt reprezentuje jeden z **256 możliwych materiałów**, gdzie wartość 0 oznacza brak voxela,

Dzięki temu uzyskano **pięciokrotną** (a w praktyce ośmiokrotną przez padding) redukcję pamięci na voxel, co pomogło również w otrzymaniu lepszej lokalności pamięci cache skutkującej w szybszych operacjach tworzenia siatek i ich edycji. Dla przykładowego świata stworzonego z 100 chunków 64x64x64 udało się zredukować zajmowaną przez voxela pamięć z **209 MB do 26 MB**.

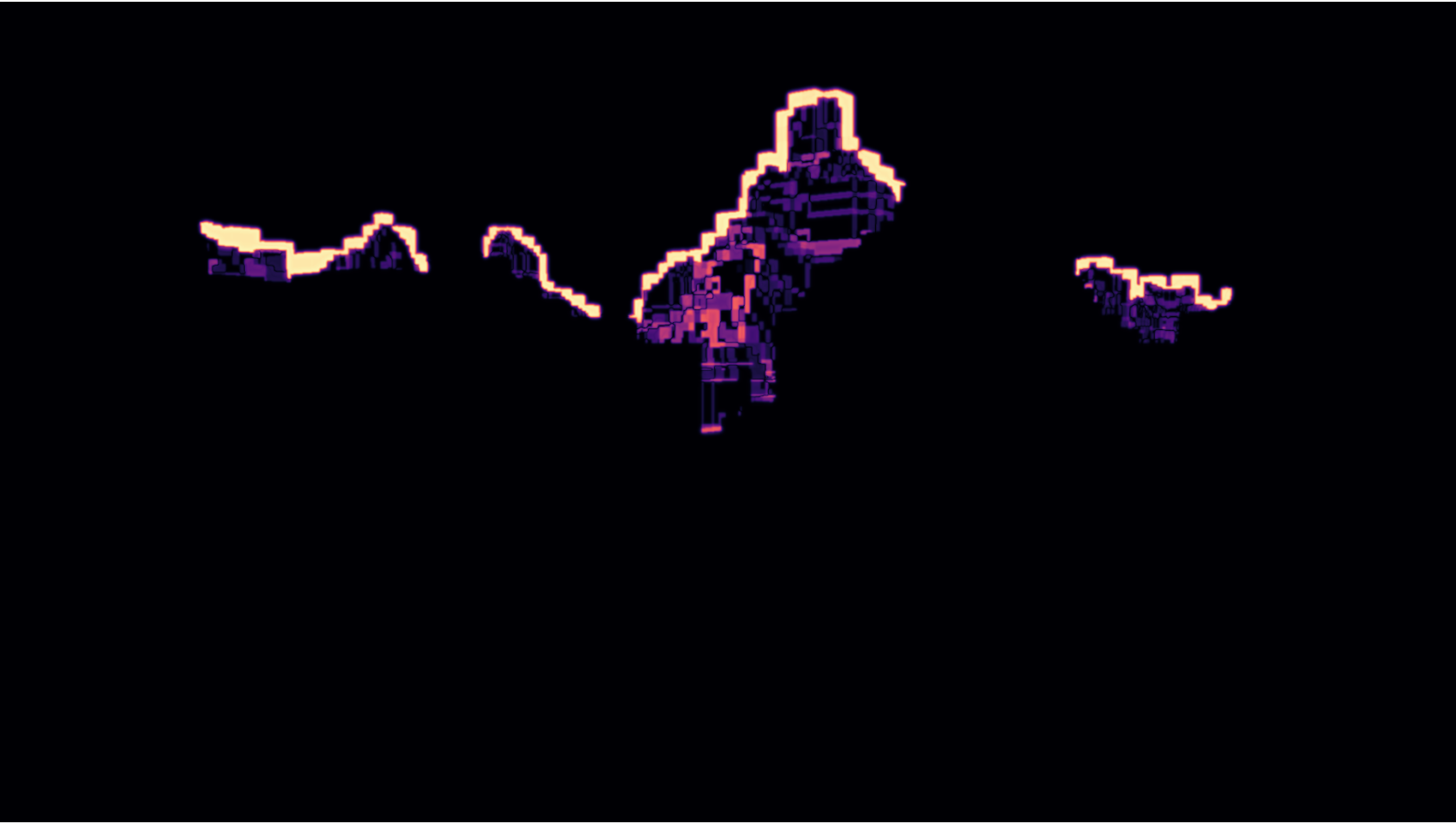
Optymalizacja pełnej sceny



Greedy Meshing



Greedy meshing + LOD



Percepcja zmian w obrazie dla ludzkiego obserwatora (FLIP)

Metoda	FPS	Trójkąty	Czas generacji chunka
Face culling	280	24.3 mln	99.86 ms
Greedy meshing	590	6.9 mln	157.84 ms
Heightmap + LOD	930	0.65 mln	11.27 ms

Połączenie **generacji terenu opartej na heightmapie** z **LOD** zapewniło najlepszy kompromis między liczbą trójkątów, czasem generacji chunków oraz liczbą FPS dla sceny ze 100 chunkami 128x128x256 (420M voxeli). Porównanie obrazów (FLIP mean = 0.016674) wskazuje na bardzo małą różnicę percepcyjną pomiędzy obrazami, co sugeruje, że zmiany wprowadzone przez LOD są praktycznie niezauważalne dla ludzkiego obserwatora.

Wnioski

- **Usuwanie ścian wewnętrznych** znacząco redukuje złożoność siatki już przy prostej implementacji.
- **Greedy meshing** zapewnia największą redukcję geometrii i najwyższy wzrost wydajności.
- Połączenie odpowiedniej techniki meshingu, chunkingu i przetwarzania równoległego umożliwia renderowanie dużych scen w czasie rzeczywistym.
- Połączenie wszystkich zastosowanych technik optymalizacji pozwoliło zwiększyć wydajność renderowania nawet około **20-krotnie** w porównaniu do implementacji naiwnej oraz zmniejszyć zużycie pamięci około **8-krotnie**.

Metody renderingu

1. Lysenko, M. (2012). *Meshing in a Minecraft game*. Ofps.net (Data ostatniej modyfikacji: 30 czerwca 2012.)
2. Lysenko, M. (2012). *An analysis of Minecraft-like engines*. Ofps.net (Data ostatniej modyfikacji: 14 stycznia 2012.)
3. Lysenko, M. (2018). *A Level of Detail method for blocky voxels*. Ofps.net (Data ostatniej modyfikacji: 3 marca 2018.)
4. Fogleman, M. (2017). *Voxel rendering techniques*. medium.com (Data ostatniej modyfikacji: 4 lutego 2017.)

Demo reel



Scan the QR code to watch the demo

